Coding Problem 1: The "Global Airline Passenger Manifest"

Scenario Background:

You are building the core backend systems for *SkyNet Airlines*. The airline needs a highly efficient system to manage the daily passenger manifest across all its flights globally.

The system handles tens of thousands of passengers a day. Security and speed are paramount. Passengers are identified by a unique Passenger object. The airline frequently needs to:

1. Check if a passenger is banned from flying (No-Fly List) almost instantly.
2. Store a list of passengers currently booked on a specific flight, maintaining the exact order in which they checked in.
3. Quickly look up which flight a specific passenger is currently assigned to across the entire global network.

Implementation Constraints & Requirements:

1. The Passenger Class (Hashing Mechanics):

- Create a class called Passenger. It must contain:
    - String passportNumber
    - String fullName
    - String nationality
- **The Hashing Constraint:** You MUST properly override both the equals() and hashCode() methods. Two Passenger objects are considered identical *if and only if* their passportNumber and nationality are the exact same. The fullName can change (e.g., due to marriage) and should *not* be part of the identity or hash calculation.

2. The Data Structures (Set, List, Map):

You must create a ManifestManager class containing three specific data structures, utilizing them precisely for their algorithmic strengths:

- **The No-Fly List (Set):**
    - Implement a globalNoFlyList using a Set<Passenger>.
    - *Constraint:* Lookups against this list must operate in average $O(1)$ time complexity.
- **The Flight Roster (List):**
    - Create a method void checkInPassenger(String flightNumber, Passenger p).
    - Store the passengers for each flight. The data structure must maintain the exact chronological order of check-in.
    - *Constraint:* You must use a List<Passenger> to store the passengers for a single flight.

- **The Global Lookup (Map):**
    - Implement a globalPassengerDirectory using a Map<Passenger, String>. The Key is the Passenger, and the Value is the flightNumber.
    - *Constraint:* Lookups to find a passenger's flight must operate in average $O(1)$ time complexity.

3. Required Operational Methods in ManifestManager:

- public void addToNoFlyList(Passenger p): Adds the passenger to the restricted set.
- public boolean processCheckIn(String flightNumber, Passenger p):
    - First, check the globalNoFlyList. If the passenger is on it, reject the check-in (return false).
    - If they are cleared, add them to the Flight Roster (List) for that specific flightNumber.
    - Add them to the globalPassengerDirectory (Map).
    - Return true if successful.
- public String locatePassengerFlight(Passenger p): Returns the flight number the passenger is currently checked into, or null if they are not checked in anywhere.

Why this makes a great interview/study question:

1. **The equals() and hashCode() Contract:** This is the most critical part of the question. By mandating that fullName is *excluded* from the identity calculation, the developer must manually override the methods. If they rely on the default object memory address, or if they include the name, the final lookup test (searchAlice) will fail and return null, creating a severe bug in the airline's system.
2. **Algorithmic Intent (Choosing the Right Collection):**
    - Why a Set for the No-Fly list? Because we only care about *existence*, and we need $O(1)$ lookups. A List would require $O(N)$ scanning.
    - Why a List for the flight roster? Because the scenario explicitly stated we must preserve the *chronological order* of check-in. A Set or Map does not guarantee insertion order (unless specifically using LinkedHashSet/LinkedHashMap).
    - Why a Map for the global lookup? Because we need to map a Key (Passenger) to a Value (Flight Number) instantly without iterating through every single flight roster.</Passenger,>

Coding Problem 2: The "MediCore" Emergency Triage System

Scenario Background:

You are developing the patient intake software for the *MediCore* hospital network. When a mass casualty event occurs, dozens of patients flood the emergency room. The system must automatically sort incoming patients so doctors always treat the most critical individuals first.

Patients have a severity level (e.g., CRITICAL, URGENT, STABLE). However, if two patients have the exact same severity, the system must prioritize the patient who arrived first (chronological order).

Implementation Constraints & Requirements:

1. The Patient Class (Comparable Contract):

- Create an enum TriageLevel with values CRITICAL, URGENT, and STABLE.
- Create a Patient class containing: String name, TriageLevel severity, and long arrivalTime.
- **The Sorting Constraint:** The Patient class MUST implement Comparable<Patient>. You must override compareTo() to enforce the two-tier sorting logic: first by severity (CRITICAL is highest priority), and then by arrivalTime (smaller timestamp is higher priority).

2. The Data Structure (Heaps/PriorityQueue):

- Create a TriageManager class.
- **The Structure Constraint:** You must use a PriorityQueue<Patient> to manage the waiting room.
- Lookups for the next patient to treat must operate in $O(1)$ time, and adding a new patient must operate in $O(\log N)$ time.

3. Required Methods:

- public void admitPatient(Patient p): Adds the patient to the queue.
- public Patient getNextPatient(): Removes and returns the highest-priority patient.

Why this makes a great interview/study question:

1. **The Comparable Contract:** It forces developers to write multi-tiered sorting logic. Many candidates struggle to sort by multiple fields without writing deeply nested if/else blocks. Utilizing Long.compare() and the enum's natural compareTo() demonstrates clean, idiomatic Java.
2. **Algorithmic Intent:** Standard List arrays require an $O(N \log N)$ sort every time a patient is added to maintain order. A PriorityQueue inserts in $O(\log N)$ by bubbling the patient up a Binary Heap, which is vastly more efficient for dynamic, continuous intake systems.

Coding Problem 3: The "TradeX" Order Matching Engine

Scenario Background:

You are building *TradeX*, a cryptocurrency exchange. The platform maintains an "Order Book" for different cryptocurrencies (e.g., BTC, ETH).

During volatile market swings, hundreds of different server threads will attempt to place orders for the exact same cryptocurrency at the exact same millisecond. If your data structures are not thread-safe, the order book will suffer from ConcurrentModificationException crashes, or worse, lose users' financial orders entirely due to race conditions.

Implementation Constraints & Requirements:

1. Concurrency Collections:

- You must create an ExchangeManager class.
- **The Map Constraint:** You must map a ticker symbol (String) to a list of orders. You are strictly forbidden from using a standard HashMap. You must use a ConcurrentHashMap.
- **The List Constraint:** The list of orders for a specific ticker must also be thread-safe for highly concurrent reads and writes. You must use a CopyOnWriteArrayList<Order>.

2. Thread-Safe Computations:

- Create a method public void placeOrder(String ticker, Order order).
- **The Initialization Constraint:** When adding an order for a *new* ticker that doesn't exist in the map yet, you must avoid Race Conditions. You cannot use a simple if (map.containsKey(ticker)) block, as another thread might insert the key between your check and your put. You MUST use the atomic computeIfAbsent() method.

Why this makes a great interview/study question:

1. **Thread-Safe Tooling:** Junior developers will slap the synchronized keyword on the entire placeOrder method. This locks the *entire exchange*—meaning someone buying ETH has to wait for someone buying BTC. ConcurrentHashMap and computeIfAbsent demonstrate senior-level lock-free (or lock-striped) thread safety.
2. **The "Check-Then-Act" Trap:** if(!map.containsKey()) { map.put() } is the most common cause of lost data in multithreaded Java applications. Forcing computeIfAbsent tests if the candidate understands atomic operations.

Coding Problem 4: The "StreamFlix" Memory Cache (LRU Mechanics)

Scenario Background:

You are working on the backend for *StreamFlix*. Fetching a movie's metadata (cast, synopsis, ratings) from the database takes 200ms. To speed this up, you want to cache the metadata in RAM.

Because RAM is highly limited, the cache can only hold exactly 100 movies. Once the cache hits 101 movies, it must automatically delete the movie that hasn't been watched in the longest time (Least Recently Used - LRU).

Implementation Constraints & Requirements:

1. The Collections API Shortcut:

- You do not need to build a Doubly-Linked List and a HashMap from scratch.
- **The Constraint:** You MUST create a class VideoCache<K, V> that extends LinkedHashMap<K, V>.

2. Access Order vs. Insertion Order:

- By default, LinkedHashMap maintains the order items were *inserted*.
- **The Constraint:** You must configure the super constructor to use **Access Order**, meaning every time an item is read via .get(), it gets moved to the back of the line automatically.

3. The Eviction Policy:

- **The Constraint:** You must override the removeEldestEntry(Map.Entry<K, V> eldest) method provided by LinkedHashMap. The method must return true whenever the size() of the map exceeds the defined capacity.

Why this makes a great interview/study question:

1. **Deep API Knowledge:** Implementing an LRU cache from scratch (HashMap + Custom Doubly Linked List) is a classic LeetCode problem. However, in enterprise Java, reinventing the wheel is a liability. Knowing that LinkedHashMap has a built-in accessOrder flag and a protected removeEldestEntry method proves deep familiarity with the JDK collections framework.
2. **Encapsulation via Inheritance:** It demonstrates how to properly extend a base collection to bake in business logic (capacity limits) seamlessly.

Coding Problem 5: The "Global E-Commerce" Sales Analyzer

Scenario Background:

You are processing a massive batch of financial data for an e-commerce platform. You are given a List of thousands of Transaction objects.

The finance team needs a highly specific report: they need to know the total revenue generated, but *only* from transactions that were marked as "COMPLETED" and occurred in the "ELECTRONICS" category.

Implementation Constraints & Requirements:

1. The Functional Paradigm:

- Create a SalesAnalyzer class with a method public double calculateElectronicsRevenue(List<Transaction> transactions).
- **The Strict Constraint:** You are explicitly forbidden from using for loops, while loops, or if statements.

2. Java 8+ Streams:

- You MUST use the Stream API.
- Use .filter() to isolate "COMPLETED" and "ELECTRONICS" transactions.
- Use .mapToDouble() to extract the financial values.
- Use .sum() to aggregate the final result.

Why this makes a great interview/study question:

1. **Declarative vs. Imperative:** A junior developer will write a massive for loop with nested if blocks and a mutable double total = 0; variable outside the loop. This problem tests the candidate's transition into modern, declarative Java.
2. **Performance Implications:** By enforcing the Stream API, the candidate demonstrates an architecture that can be trivially upgraded to handle 50 million rows. Changing .stream() to .parallelStream() would instantly multithread this calculation, something that is incredibly difficult to do safely with imperative for loops.</K,></K,></K,>

Problem 1: The LRU Cache Eviction Engine (LinkedHashMap)

Enterprise Scenario: An e-commerce backend relies on an in-memory cache to store product details for extremely fast O(1) lookups. However, server RAM is limited. You must implement a cache that only holds a maximum of 5 items. If a 6th item is added, the cache must automatically evict the "Least Recently Used" (LRU) item to prevent an OutOfMemoryError.

Task: Extend Java's LinkedHashMap to build this LRU Cache. You must configure the constructor to use *Access-Order* (not insertion-order) and override the protected removeEldestEntry method. Demonstrate its behavior by adding 5 items, accessing the first item, adding a 6th item, and proving the 2nd item was the one evicted.

Code Breakdown & Memory Analysis:

- super(capacity, 0.75f, true): Setting the third parameter to true switches the internal Doubly-Linked List from tracking *insertion* order to tracking *access* order. Every time .get() or .put() is called, that specific node is disconnected from its current position and physically re-linked to the tail of the list.
- removeEldestEntry: The LinkedHashMap calls this method internally after every .put(). By returning size() > capacity, we instruct the map to sever the Head node (the oldest/least accessed item) for Garbage Collection.

Problem 2: The "Lost in the Map" Memory Leak (Mutable Keys)

Enterprise Scenario: A junior developer used a custom Session object as a key in a HashMap to store user authorization tokens. After inserting the session into the map, the developer updated the session's timestamp. Suddenly, the application can no longer retrieve the tokens, and the server's Heap memory is slowly filling up with unretrievable zombie objects.

Task: Write a program that deliberately replicates this bug. Create a mutable SessionKey class with a properly overridden equals() and hashCode(). Insert it into a map, mutate the key, and attempt to retrieve the value. Explain the exact hashing failure occurring in memory.

Code Breakdown & Memory Analysis:

- Output yields Token after mutation: null.
- **The Hashing Failure:** When .put() was called, hashCode() generated a number based on "USER_123" and placed the node in Bucket 5.
- When the field was mutated to "USER_456", the key's internal state changed, meaning its hashCode() also changed.
- When .get() is called, the Map calculates the *new* hash and searches Bucket 12. Finding nothing there, it returns null. The original node is permanently lost in Bucket 5, causing a massive memory leak. **Map keys must always be immutable (final).**

Problem 3: The Thread-Safe Sweeper (ConcurrentModificationException)

Enterprise Scenario: An e-commerce cart cleanup job needs to loop through a user's active shopping cart and remove any items that are completely out of stock in the database. A standard for-each loop is crashing the entire service.

Task: Write a program that iterates through an ArrayList<String> of products. If a product equals "Out_Of_Stock", remove it. Deliberately trigger a ConcurrentModificationException first, catch it to print an error log, and then execute a safe wipe using an explicit Iterator.

Code Breakdown & Memory Analysis:

- cart.remove(item) inside the for-each loop modifies the collection's size (structural modification). The internal modCount variable increments. The hidden iterator powers the for-each loop checks this modCount. Seeing it altered unexpectedly, it throws a "Fail-Fast" exception to prevent unpredictable indexing bugs.
- iterator.remove() works safely because it updates the internal modCount simultaneously for both the ArrayList and the Iterator, keeping the traversal synchronized and avoiding array-shifting index skips.

Problem 4: Advanced Aggregation & Multi-Tier Sorting

Enterprise Scenario: A data analytics dashboard receives a massive, unsorted list of Employee Transaction objects. You must aggregate the total transaction amounts per employee, then sort the results to find the highest earners. If two employees have the exact same total, sort them alphabetically by name.

Task: Take an unsorted List<Transaction>. Use a HashMap to aggregate total amounts by employee name. Transfer the Map entries into a List<Map.Entry<String, Double>>. Sort this list using a custom Comparator that handles the descending numeric sort and the ascending alphabetical tie-breaker.

Code Breakdown & Memory Analysis:

- getOrDefault(): Extremely efficient O(1) lookup that avoids nested if-else null checks during aggregation.
- **The Comparator Logic:** Returning negative, positive, or zero determines the swap. If amountCompare == 0, it triggers the String's native .compareTo() method, perfectly resolving the tie-breaker structurally.

Problem 5: Forcing a Hash Collision (Internal Mechanics)

Enterprise Scenario: An attacker is trying to execute a Denial of Service (DoS) attack on your server by submitting massive JSON payloads containing keys engineered to have the exact same HashCode, forcing the HashMap to degrade from O(1) to O(n) performance.

Task: Create a MaliciousKey class that purposefully returns the exact same integer 1 for *every* instance's hashCode(), but properly differentiates equality using an internal ID. Insert 10 instances into a HashMap. Explain what happens inside the JVM's specific memory bucket, and describe how Java 8 protects against this via Treeification.

Code Breakdown & Memory Analysis:

- Because hashCode() constantly returns 1, the bitwise math (n-1) & hash forces every single node into Bucket 1.
- The map checks equals(). Since ID_1 != ID_2, it attaches the new node to the end of a long Linked List in Bucket 1.
- **Performance Degradation:** Looking up ID_9 now takes O(n) time because the JVM must linearly traverse 9 nodes to find the matching string.
- **Java 8 Treeification Rescue:** Once the Linked List in Bucket 1 reaches 8 nodes (and the array capacity grows past 64), the JVM intercepts the attack. It destroys the Linked List and dynamically restructures the nodes into a **Red-Black Tree**. This forces the search time back down to a highly optimized O(\log n), preventing the server CPU from stalling under the heavy load.